

Assignment2: Deploying PyTorch ML Workloads with Docker & Kubernetes

Course: MLOps & Infrastructure for Machine Learning

Duration: Till End-sem **Type:** Individual Assignment

Overview

In this assignment, you will take a PyTorch image classification model through the full deployment lifecycle: from local development with proper Git workflows, to containerized training with Docker, to orchestrated deployment on Kubernetes. By the end, you will have a production-style ML pipeline that can train and serve predictions at scale.

Learning Objectives

By completing this assignment, you will be able to:

- 1 Structure an ML project repository with proper Git practices (branching, PRs, `.gitignore`, secrets management)
- 2 Write multi-stage Dockerfiles optimized for ML workloads
- 3 Deploy PyTorch training jobs on Kubernetes using Jobs and persistent storage
- 4 Serve a trained model via a Kubernetes Deployment with health checks
- 5 Use ConfigMaps and Secrets for environment-specific configuration

Prerequisites

- Python 3.10+, PyTorch experience
 - Docker Desktop installed (or access to a Docker-enabled VM)
 - `kubectl` CLI installed
 - A Kubernetes cluster (Minikube, kind, or a cloud-managed cluster)
 - A GitHub account
-

Part A: Repository Setup (15 points)

- 1 **Create a public GitHub repository** named `mlops-pytorch-pipeline`.
- 2 **Initialize the project** with the following structure:

```
mlops-pytorch-pipeline/
... README.md
... .gitignore
... .github/
.   ... workflows/
.       ... ci.yml
... src/
.   ... train.py
.   ... model.py
.   ... dataset.py
.   ... serve.py
... configs/
.   ... training_config.yaml
```

```

... docker/
.   ... Dockerfile.train
.   ... Dockerfile.serve
... k8s/
.   ... namespace.yaml
.   ... training-job.yaml
.   ... serving-deployment.yaml
.   ... serving-service.yaml
.   ... configmap.yaml
.   ... hpa.yaml
... requirements/
.   ... train.txt
.   ... serve.txt
... tests/
.   ... test_model.py

```

3 Git workflow requirements:

- Create a `develop` branch from `main`
- All work must be done on feature branches (e.g., `feature/docker-training`, `feature/k8s-deployment`)
- Each feature branch must be merged via a Pull Request with a meaningful description
- Minimum 2 PRs for Week 1, minimum 2 PRs for Week 2
- Write meaningful commit messages following Conventional Commits

Part B: PyTorch Model (10 points)

Implement a simple image classifier using PyTorch:

- **Model** (`src/model.py`): A CNN or fine-tuned `torchvision` model (e.g., ResNet-18) for CIFAR-10 or Fashion-MNIST
- **Dataset** (`src/dataset.py`): Data loading with `torchvision.datasets`, including transforms and `DataLoader` setup
- **Training** (`src/train.py`): Training loop that:
 - Reads hyperparameters from `configs/training_config.yaml`
 - Logs metrics (loss, accuracy) to stdout in structured format (JSON lines)
 - Saves the trained model checkpoint to a configurable output path
 - Supports early stopping
- **Serving** (`src/serve.py`): A Flask or FastAPI app that:
 - Loads a saved model checkpoint
 - Exposes `POST /predict` (accepts an image, returns class probabilities)
 - Exposes `GET /health` (returns 200 if model is loaded)

Part C: Docker Containerization (30 points)

Training Image (`docker/Dockerfile.train`)

Write a multi-stage Dockerfile for the training workload:

```
# Example structure · you must complete and optimize this
```

```

# Stage 1: Base with dependencies
FROM python:3.11-slim AS base
WORKDIR /app
COPY requirements/train.txt .
RUN pip install --no-cache-dir -r train.txt
# Stage 2: Training runtime
FROM base AS training
COPY src/ ./src/
COPY configs/ ./configs/
ENV PYTHONUNBUFFERED=1
ENTRYPOINT ["python", "src/train.py"]

```

Requirements:

- Pin all dependency versions in `requirements/train.txt`
- The training script must read its config from a mounted volume or environment variable

Serving Image (docker/Dockerfile.serve)

Write a Dockerfile for model serving:

Requirements:

- Based on a slim Python image
- Installs only inference dependencies (no training libraries like tensorboard)
- Exposes port 8080
- Runs as a non-root user
- Includes a HEALTHCHECK instruction

Verification

Demonstrate that the following works locally:

```

# Build training image
docker build -f docker/Dockerfile.train -t mlops-train:v1 .
# Run training with mounted volumes
docker run --rm \
  -v $(pwd)/data:/app/data \
  -v $(pwd)/checkpoints:/app/checkpoints \
  mlops-train:v1
# Build serving image
docker build -f docker/Dockerfile.serve -t mlops-serve:v1 .
# Run serving
docker run --rm -p 8080:8080 \
  -v $(pwd)/checkpoints:/app/checkpoints \
  mlops-serve:v1
# Test prediction endpoint
curl -X POST http://localhost:8080/predict \
  -F "image=@test_image.png"

```

Include a screenshot or terminal log of these commands succeeding in your PR description.

Bonus: Kubernetes Deployment

Part D: Kubernetes Training Job (10 points)

Create Kubernetes manifests to run training as a Job:

k8s/namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: ml-training
  labels:
    project: mlops-pytorch-pipeline
```

k8s/configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: training-config
  namespace: ml-training
data:
  training_config.yaml: |
    model:
      architecture: resnet18
      num_classes: 10
    training:
      epochs: 10
      batch_size: 64
      learning_rate: 0.001
      early_stopping_patience: 3
    data:
      dataset: cifar10
      data_dir: /app/data
    output:
      checkpoint_dir: /app/checkpoints
      model_name: classifier_v1.pt
```

k8s/training-job.yaml

Write a Kubernetes Job manifest that:

- Uses your training Docker image
- Mounts the ConfigMap as a volume at /app/configs
- Uses a PersistentVolumeClaim for /app/data and /app/checkpoints
- Sets resource requests and limits (CPU: 2 cores, Memory: 4Gi)

Bonus (5 points): Add GPU resource requests (nvidia.com/gpu: 1) with a node selector or toleration for GPU nodes.

Part E: Kubernetes Model Serving (5 points)

k8s/serving-deployment.yaml

Write a Deployment manifest that:

- Runs 2 replicas of the serving container
- Mounts the checkpoint PVC (read-only) at `/app/checkpoints`
- Configures:
 - **Liveness probe:** GET `/health` every 10s, failure threshold 3
 - **Readiness probe:** GET `/health` every 5s, initial delay 15s
- Sets resource requests (CPU: 500m, Memory: 1Gi) and limits (CPU: 1, Memory: 2Gi)
- Uses a rolling update strategy (`maxSurge: 1, maxUnavailable: 0`)

`k8s/serving-service.yaml`

Create a Service of type `ClusterIP` (or `LoadBalancer` if using a cloud cluster) exposing port 80 targeting container port 8080.

Part F: End-to-End Validation (10 points)

Demonstrate the full workflow running on your Kubernetes cluster:

- 1 Apply all manifests:

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/training-job.yaml
```

- 2 Once training completes, deploy the serving layer:

```
kubectl apply -f k8s/serving-deployment.yaml
kubectl apply -f k8s/serving-service.yaml
kubectl apply -f k8s/hpa.yaml
```

- 3 Verify pods are running and healthy:

```
kubectl get pods -n ml-training
kubectl describe deployment model-serving -n ml-training
```

- 5 Test the prediction endpoint:

```
# Port-forward for local testing
kubectl port-forward svc/model-serving 8080:80 -n ml-training
# Send a prediction request
curl -X POST http://localhost:8080/predict \
  -F "image=@test_image.png"
```

Deliverable: Include terminal output or screenshots of each step in your final PR.

Submission Instructions

- 1 Ensure all code is merged to the `main` branch via PRs
- 2 Your repository must have:
 - A clear `README.md` with setup instructions and architecture diagram
 - At least 4 merged PRs (2 per week) with meaningful descriptions
- 3 Submit the following on the course platform:

- Link to your GitHub repository
 - Link to your final PR that includes validation screenshots
 - A short write-up (300-500 words) reflecting on:
 - What was the most challenging part?
-

Resources

- Docker Best Practices for Python
 - Kubernetes Jobs Documentation
 - PyTorch Deployment with TorchServe
 - GitHub Actions for Python
-

Starter Code Snippets

The following snippets provide a starting point. You are expected to extend, improve, and integrate them into the full project.

src/dataset.py

```
import torch
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

def get_transforms(train: bool = True) -> transforms.Compose:
    if train:
        return transforms.Compose([
            transforms.RandomHorizontalFlip(),
            transforms.RandomCrop(32, padding=4),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.4914, 0.4822, 0.4465],
                std=[0.2470, 0.2435, 0.2616],
            ),
        ])
    return transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.4914, 0.4822, 0.4465],
            std=[0.2470, 0.2435, 0.2616],
        ),
    ])

def get_dataloaders(
    data_dir: str,
    batch_size: int = 64,
    num_workers: int = 2,
) -> tuple[DataLoader, DataLoader]:
    train_dataset = datasets.CIFAR10(
        root=data_dir,
        train=True,
```

```

        download=True,
        transform=get_transforms(train=True),
    )
    val_dataset = datasets.CIFAR10(
        root=data_dir,
        train=False,
        download=True,
        transform=get_transforms(train=False),
    )
    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True,
        num_workers=num_workers,
        pin_memory=True,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=num_workers,
        pin_memory=True,
    )
    return train_loader, val_loader

```

src/train.py

```

import json
import sys
from pathlib import Path
import torch
import torch.nn as nn
import yaml
from dataset import get_dataloaders
from model import get_model
def load_config(config_path: str) -> dict:
    with open(config_path) as f:
        return yaml.safe_load(f)
def train_one_epoch(
    model: nn.Module,
    loader: torch.utils.data.DataLoader,
    optimizer: torch.optim.Optimizer,
    criterion: nn.Module,
    device: torch.device,
) -> tuple[float, float]:
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0
    for batch_idx, (inputs, targets) in enumerate(loader):
        inputs, targets = inputs.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)

```

```

        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * inputs.size(0)
        _, predicted = outputs.max(1)
        total += targets.size(0)
        correct += predicted.eq(targets).sum().item()
    avg_loss = total_loss / total
    accuracy = correct / total
    return avg_loss, accuracy
@torch.no_grad()
def evaluate(
    model: nn.Module,
    loader: torch.utils.data.DataLoader,
    criterion: nn.Module,
    device: torch.device,
) -> tuple[float, float]:
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    for inputs, targets in loader:
        inputs, targets = inputs.to(device), targets.to(device)
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        total_loss += loss.item() * inputs.size(0)
        _, predicted = outputs.max(1)
        total += targets.size(0)
        correct += predicted.eq(targets).sum().item()
    avg_loss = total_loss / total
    accuracy = correct / total
    return avg_loss, accuracy
def main():
    config_path = Path("/app/configs/training_config.yaml")
    if not config_path.exists():
        config_path = Path("configs/training_config.yaml")
    config = load_config(str(config_path))
    device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
    model = get_model(
        architecture=config["model"]["architecture"],
        num_classes=config["model"]["num_classes"],
    ).to(device)
    train_loader, val_loader = get_dataloaders(
        data_dir=config["data"]["data_dir"],
        batch_size=config["training"]["batch_size"],
    )
    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=config["training"]["learning_rate"],
    )
    criterion = nn.CrossEntropyLoss()
    best_val_loss = float("inf")
    patience_counter = 0

```



```

patience = config["training"]["early_stopping_patience"]
checkpoint_dir = Path(config["output"]["checkpoint_dir"])
checkpoint_dir.mkdir(parents=True, exist_ok=True)
for epoch in range(config["training"]["epochs"]):
    train_loss, train_acc = train_one_epoch(
        model, train_loader, optimizer, criterion, device
    )
    val_loss, val_acc = evaluate(model, val_loader, criterion,
device)
    log_entry = {
        "epoch": epoch + 1,
        "train_loss": round(train_loss, 4),
        "train_accuracy": round(train_acc, 4),
        "val_loss": round(val_loss, 4),
        "val_accuracy": round(val_acc, 4),
    }
    print(json.dumps(log_entry), flush=True)
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        save_path = checkpoint_dir /
config["output"]["model_name"]
        torch.save({
            "epoch": epoch + 1,
            "model_state_dict": model.state_dict(),
            "optimizer_state_dict": optimizer.state_dict(),
            "val_loss": val_loss,
            "val_accuracy": val_acc,
        }, save_path)
        print(json.dumps({"event": "checkpoint_saved", "path":
str(save_path)}), flush=True)
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(json.dumps({"event": "early_stopping",
"epoch": epoch + 1}), flush=True)
            break
        print(json.dumps({"event": "training_complete",
"best_val_loss": round(best_val_loss, 4)}), flush=True)
if __name__ == "__main__":
    main()

```
